

■ POC-AI-AGENTS · DISEÑO DE ARQUITECTURA

Agente autónomo + AI Firewall local — sin mocks, 100% funcional

Jira Cloud real, SonarQube real escaneando código real, un grafo Neo4j sincronizado desde Azure DevOps real y un índice RAG en Qdrant, todo expuesto a Copilot vía MCP. Ningún paso del pipeline imprime un **echo** fijo: cada etapa llama a un sistema real y dejar rastro en `logs/firewall_audit.jsonl`.

7SERVICIOS DOCKER

4SERVIDORES MCP

3SISTEMAS CON CREDENCIALES REALES

0DATOS SIMULADOS

Servicios Docker

docker compose up -d --build

neo4j:7474 :7687 Grafo de dependencias. Healthcheck real vía cypher-shell. NEO4J:5.12.0-COMMUNITY	graph-seederefímero Fallback idempotente (MERGE) si aún no sincronizaste desde Azure DevOps. DEPENDS_ON: HEALTHY	sonarqube:9000 Servidor real de análisis estático. SONARQUBE:10.6-COMMUNITY	sonar-scannerefímero Rota password admin, genera token, escanea los 3 módulos de sample-repo/. BOOTSTRAP_SONAR.SH
qdrant:6333 Vector store para RAG: código, histórico Jira, hallazgos Sonar resueltos. QDRANT/QDRANT	rag-indexerefímero Embeddings locales (MiniLM) → upsert a las 3 colecciones de Qdrant. INDEX_RAG_CORPUS.PY	ai-firewall:8080 FastAPI real: ingress jailbreak, egress redacción, auditoría persistente. FIREWALL_PROXY.PY	

Orquestador de contexto: grafo + RAG vía MCP

registrados en `.vscode/mcp.json` — Copilot Chat los invoca nativamente, no bash

Azure DevOps (real) Repos y work items de tu organización — puebla el grafo, no un seed fijo.
Jira Cloud (real, OAuth) Copilot busca y lee otros tickets/comentarios por su cuenta, más allá del que arma el prompt.
sample-repo/ código Java, TypeScript, Python — embebido para búsqueda semántica.
Histórico Jira + Sonar resuelto Tickets pasados y vulnerabilidades ya remediadas, vía API real.



■ @azure-devops/mcp

repos · work items

■ atlassian (mcp.atlassian.com)

búsqueda Jira · OAuth

■ mcp-neo4j-cypher

read / write cypher

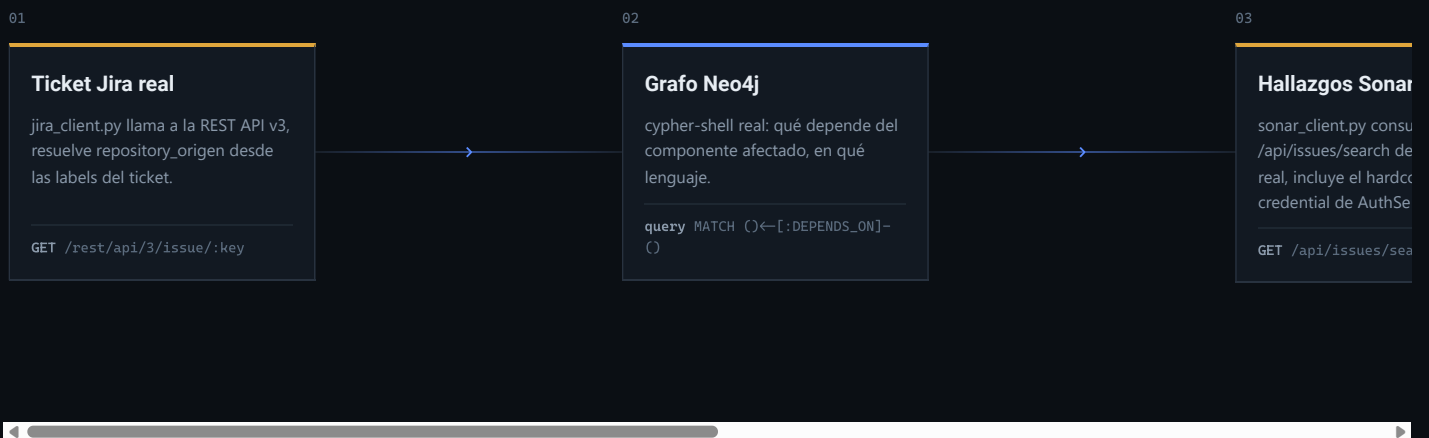
■ mcp-server-qdrant

qdrant-find / qdrant-store

jira_client.py sigue siendo el que alimenta run_poc_loop.sh (llamada REST determinística) – el MCP atlassian es para exploración interactiva de Copilot Chat, no reemplaza al cliente del pipeline.

Pipeline de run_poc_loop.sh

sin argumentos – el escenario lo decide el ticket real de Jira. Orquestación a demanda, no CI/CD: nada dispara esto automáticamente cuando aparece un commit nuevo – ver nota en "Estructura del repositorio" más abajo.



Lógica de decisión del firewall

orden estricto – egress solo corre si ingress pasa

PASO 1

Ingress · Jailbreak

texto normalizado contiene `rm -rf` , `drop table` , `ignore previous instructions` ... ⇒ **403 REJECTED**

– corta el pipeline, gh copilot nunca se invoca

PASO 2

Egress · Fuga de datos

regex sobre el prompt real: `password=` , `secret_key=` , patrón Azure key ⇒ **redactions_applied ≥ 1**

reemplazo por [REDACTED_CORPORATE_SECRET]

PASO 3

Veredicto final

200 APPROVED sanitized_prompt no nulo → gh copilot explain real

Cache local con TTL

cache_utils.py - evita re-golpear Jira/Sonar en cada corrida

```
./cache/<hash>.json · TTL 300s (CACHE_TTL_SECONDS) · {"_cache": {"hit": bool, "age_seconds": n}} · jira_client.py + sonar_client.py
```

check_prereqs.sh — antes de correr el pipeline

falla rápido y claro en vez de un curl criptico a mitad de la demo

✓ Neo4j alcanzable vía cypher-shell

✓ SonarQube UP + SONAR_TOKEN válido

✓ Qdrant listo (/readyz)

✓ Credenciales Jira válidas (/myself)

✓ JIRA_TICKET_KEY definido

✓ Credenciales Azure DevOps válidas

✓ gh copilot disponible

Estructura del repositorio

```
poc-ai-agents/  
├─ docker-compose.yml  
├─ Dockerfile.firewall · requirements.txt  
├─ firewall_proxy.py  
├─ jira_client.py · sonar_client.py · cache_utils.py  
├─ sample-repo/  
│   ├── auth-service/ (Java - password hardcodedo real)  
│   ├── frontend/ (TypeScript)  
│   └─ data-worker/ (Python)  
├─ seed/seed.cypher # fallback  
├─ .vscode/mcp.json # 4 MCP  
├─ prompts/sync_graph_from_azure_devops.md  
├─ scripts/  
│   ├── bootstrap_sonar.sh  
│   ├── index_rag_corpus.py  
│   ├── check_prereqs.sh  
│   └─ report_sprint_metrics.py  
├─ cache/ · logs/ · qdrant_storage/ # runtime  
├─ .env.example · .gitignore  
├─ run_poc_loop.sh  
└─ README.md
```

- + **Jira real** — REST API v3, no JSON local; el escenario lo decide el ticket, no un flag.
- + **SonarQube real** — escanea sample-repo/ de verdad; la credencial hardcodeda en AuthService.java es un hallazgo real, no fabricado.
- + **Grafo poblado desde Azure DevOps** — vía MCP, no un seed estático (que queda solo como fallback).
- + **RAG semántico** — Qdrant + embeddings locales sobre código e histórico real de Jira/Sonar.
- + **Cache con TTL** — transparente sobre qué se sirvió en vivo vs. cacheado.
- + **check_prereqs.sh** — valida los 7 servicios y 3 credenciales antes de arrancar.
- + **Ejecución real, no solo explicación** — gh copilot suggest corre de verdad (con confirmación humana) y aplica el cambio en una rama de revisión, nunca en main.

- + **Métrica de colaboración** — copilot_contribution.jsonl + report_sprint_metrics.py miden cuánto sugirió y aplicó Copilot, no solo si el pipeline corrió.
- + **Auditoría en el ticket mismo** — jira_client.py comment publica el veredicto del firewall y qué hizo Copilot como comentario real en Jira, visible sin acceso a los logs locales.
- ⚠ **"Pipeline" ≠ CI/CD** — run_poc_loop.sh es orquestación manual a demanda. Las ramas copilot/* que crea la etapa 5 no disparan ningún build/test/re-análisis automático; nada las está observando todavía.

Auditoría y métricas de contribución

logs/firewall_audit.jsonl (decisiones) + logs/copilot_contribution.jsonl (colaboración real)

```
{"ts": "2026-07-03T14:02:11Z", "ticket_id": "JIRA-404", "status": "APPROVED", "redactions_applied": 2, "reason": null} {"ts": "2026-07-03T14:03:47Z", "ticket_id": "JIRA-404", "status": "REJECTED", "redactions_applied": 0, "reason": "jailbreak_pattern_matched"}
```

```
python3 scripts/report_sprint_metrics.py --since 2026-07-01
```

```
Corridas totales: 12 Tickets unicos tocados: 5 Aprobadas por el firewall: 10 Rechazadas por el firewall: 2 Tasa de aprobacion: 83% Copilot sugirio un cambio: 10 Sugerencia aplicada a rama: 7 Corridas con dato redactado: 3
```

PLAN.md · README.md — mismo directorio · esquema estático, sin llamadas externas